Operating systems

Oscar Palm

SP 1 2025

Contents

Lecture 1

Introduction

1.1 Introduction to C

Code snippet 1.0

```
#include <stdio.h> // Tells the preprocessor to load a file

int main()       // The entrypoint of the program
{            // Defines a block of code, everything created in it and not returned is only defined for the block
   printf ("Hello world\n");  // A basic statement
  return 0;
}
```

For this course we will use Chalmers remote StuDat computers.

1.2 Course introduction

The course book is "Modern Operating Systems", by Tanenbaum and Bos.

Vincenso should work on his standup comedy.

1.2.1 Why study OS?

Provides services to system users. Studying it gives us an overview of how computers work on a lower level.

The operating system is both there to help the user actually use the computer, but also to protect the hardware from the main source of complications; the user. It makes sure that everything that needs to run on the computer is able to do so, and is given sufficient resources to perform its task.

1.3 What is an Operating system

An operating system is a program which provides a *set of services* to system users (human users and computer programs). It also protects the user from the hardware and vice verse.

An operating system manages resources on the computer and makes sure that all system users get the necessary resources (if possible), such as cpu(s), memory to store data, i/o devices and so on. It controls the execution of different programs, makes sure that an error in one doesn't cascade into others and makes sure different programs stay independent and cannot access each others' resources.

Saving a file

When the user is saving a file, without an operating system, nothing else would be able to happen on the computer while waiting for that; with a good operating system, the hardware resources can be managed in such a way that other tasks can be performed while waiting for the file to be written.

1.4 The event-driven (interrupt-drive) life of an OS

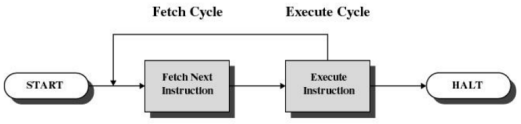


Figure 1.1: A simple fetch cycle where only one thing happens

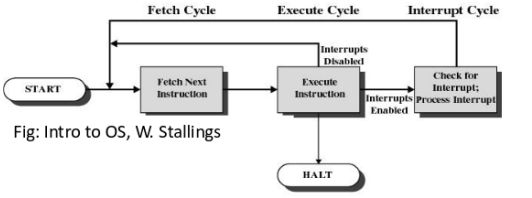


Figure 1.2: A more advanced fetch cycle where things can happen in the background

By looking for interrupts between running the instructions we are able to handle events from different parts of the system. With this we can achieve what was talked about in the example above, where other programs execute while the saving of a file was happening.
Scheduling is about choosing for how long a certain program can run before it needs to be interrupted for another program to be able to run. It is the operating system's job to choose for how long, and to choose the order the processes gets to run.

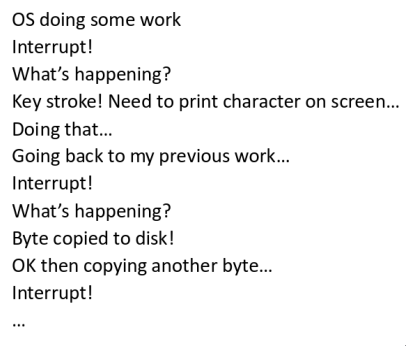


Figure 1.3: An example of a basic event cycle.

1.5 System calls (overview)

System calls are ways for user programs to *"talk to"* the hardware. All actions a user wants to do which modifies the hardware requires a system calls, e.g. Creating files, sending data over internet, showing something on a screen; are created through system calls. All system calls are types of interrupts. When the CPU finds the interrupt, control is given to the operating system, which chooses how to do the thing the user wants to do (if possible and the user should be able to).

1.6 Dual-Mode and Multimode Operation

What happens if a user tries to bypass the operating system by directly modifying the hardware without using system calls? The hardware keeps track of what kind of program is running, when the operating system is running, a bit is set which allows it to run "special functions" that normal users cannot.

To make sure the user cannot set that bit for themselves, when the hardware first starts, it loads the operating system into memory; during this phase the hardware and operating system defines which operations are privileged, and what code to run when those actions are performed. When the user tries to do those privileged actions, for example modifying a file, the operating system starts running.

1.7 Operating system services

1.8 System calls (continued) and APIs [Self reading]

1.9 System boot [Self reading]

1.10 Operating systems structures

Lecture 2

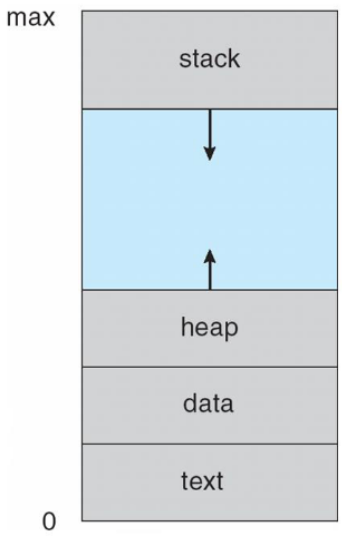
Processes

We want to be able to run several programs at the same time, but our cpu is only able to run one program at the time. This way of looking at the problem is impossible to solve. What we actually want is the feeling of several programs running at the same time, and while a cpu core cannot execute several programs at once, it can continually switch between executing several different processes without completing them. This is called content switching.

Think of the cpu as a human performing a task, if we for example are cooking and while cutting the vegetables we accidentally cut ourselves, we don't need to finish cooking before rinsing and plastering the wound, we instead stop cooking for a while to fix the higher priority task; stopping the bleeding, before continuing to cook.

2.1 Introduction to processes

A process is an execution of a program, the execution must be sequential. The program is the executable entity on disk, a process is when it runs, active. When the executable program is loaded into memory, it becomes a process. When a program is run several times, simultaneously or at different time, they are separate processes and cannot affect each others. A process can have sub processes.



Text section The program code.

Data section Data section containing global variables.

Heap section Heap containing memory dynamically allocated during run time.

Stack section Stack containing temporary data like f parameters, return addresses, local variables.

Figure 2.1: How a process' memory is allocated

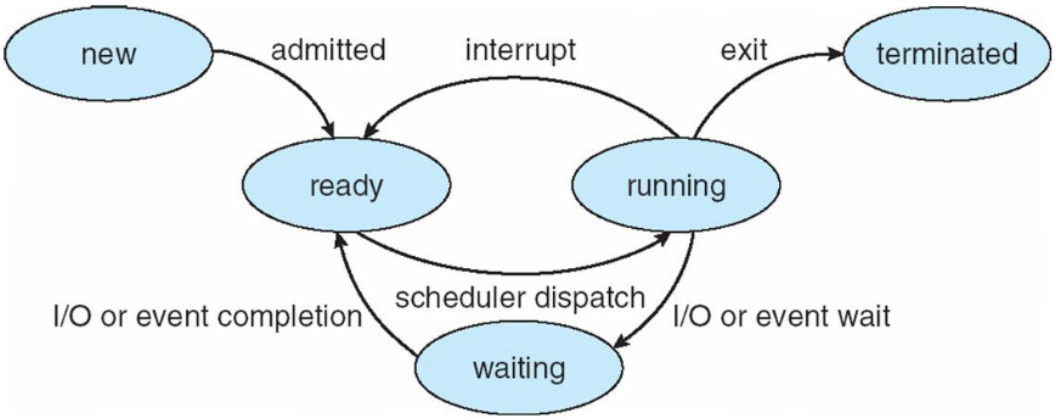
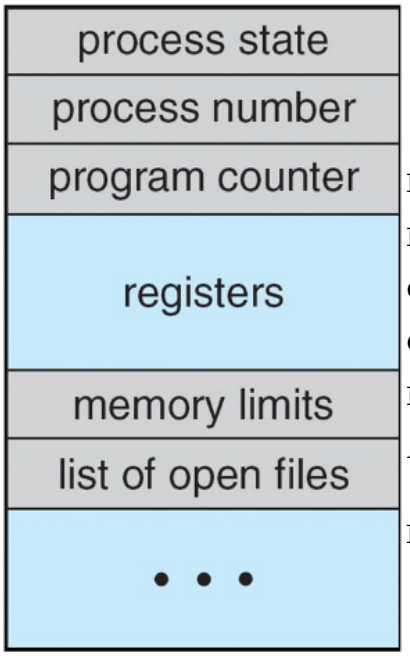


Figure 2.2: Diagram of process state

2.1.1 Context switching

To be able to switch between executing several different processes, a concept called PCB (Process Control Block) is utilized.



Process state running, waiting, etc.

Program counter location of next instruction to execute

CPU registers contents of all process-centric registers

CPU scheduling information priorities, scheduling queue pointers

Memory-management memory allocated to the process

Accounting information CPU used, clock time elapsed since start, time limits

I/O status information I/O devices allocated to process, list of open files

Figure 2.3: A process content block

2.2 Process scheduling

How we schedule which process should be executed next is a very important task, since a lot of processes are usually waiting for data from devices, it can be very time inefficient to store all processes in a list and just executing the first ready process. To speed up the execution we instead separate processes into different lists, when a process is waiting for input from a certain I/O device, we put it in a list connected to that I/O device, and when a process is waiting for the disk, we put it into a list connected to the disk, this way we limit the numbers of processes we need to check when choosing which process should be the next to run.

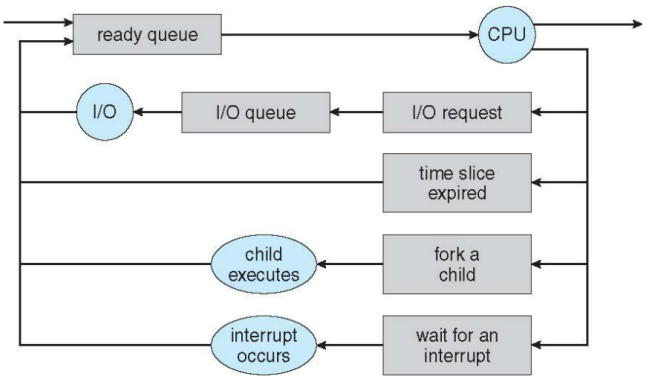


Figure 2.4: Example of how we choose next process to run

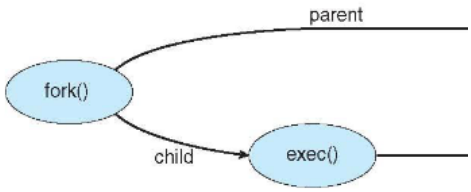
2.3 Operations on processes

2.3.1 Process creation

The user needs to be provided the ability to create and terminate processes. Since processes can't be created from nowhere, we have a higher process which can create child processes, to be able to do this, we need a system call to create processes.

From this we get a tree structure of processes, with the root process, init, always at the top, with pid = 1.

The command we use to be able to create new processes is fork(), which creates a copy of the current process, if we then want that new process to do something different than the parent program, we have a command exec(), which helps us load a new program into the process. NOTE: When calling fork(), the child and the parent will get differing results, provided nothing went wrong, the child process will get the value 0 returned while the parent gets the process ID of the created child returned.




```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```

Figure 2.5: A C program creating a child

2.3.2 Process termination

When a process is finished executing, it calls the operating system using exit(), this tells the operating system to clean up after the program, free allocated memory and stuff.

Sometimes the termination of a process cannot be completed, for example if the child process is done, trying to return its result to the parent **but** the parent hasn't waited for the response, the process cannot be completely removed.

2.4 Interprocess communication

Lecture 3

Multithread Programming

3.1 Threads (Introduction)

A computer with multiple cores is like a body with multiple brains; at the same time, multiple instructions can be executed by the cpu. These instruction can be from different processes, but we also want to be able to run different parts of the same process on several cores.

When we talk about multithreading in programming, we talk about the operating system supporting having several program counters within the same program, this means we can execute several instructions in the same program at the same time (if the operating system have more than one core), and / or execute the program in a non-linear fashion.

One of the main reasons for creating threads within a program instead of creating new processes is that creating threads is a much simpler action for the operating systems, it takes significantly less time and resources.

3.1.1 Concurrency vs Parallelism

When we talk about executing code simultaneously we usually talk about either concurrent or parallel programs, the difference is that while a parallel program is running on different **cores** and are executing at the same time, a concurrent program is one that runs in what feels like the same time, they could execute on different cores, or they could be running on the same core, meaning they will **maybe** not execute at the same time but given enough time, both will progress.

3.2 Multithreading models

3.2.1 User Threads and Kernel Threads

User threads are threads managed by the user, the operating system is not aware of these. Think Java threads or similar.
Kernel threads are threads supported by the kernel.

Kernel threads are useful e.g. when we have a program where some parts of the program which are taking a lot of time without really blocking other things we want to do, think of a process wanting to save a file, we still want to

be able to continue running the process.

User threads are useful when we have multiple threads but want more control over when they should execute.

There are several different models for how we combine user threads and kernel threads.

3.2.2 Many-to-One

Many user threads running in one kernel thread.

3.2.3 One-to-one

One user thread running in one kernel thread.

3.2.4 Many-to-many

Allowing several user threads to be mapped to several kernel threads, look at ThreadFiber for Windows.

3.3 Implicit threading

Instead of the programmer managing threads explicitly, we can use pre-made constructions such as thread pools to speed up the assignment of threads to tasks and minimize the human errors of explicit thread management.

3.4 Threading issues

Lecture 4

Process scheduling

4.1 Introduction to process scheduling

There's two different kinds of processes; CPU-bound and I/O bound.

4.1.1 CPU-bound

A cpu bound thread is a thread that is mainly using computing and not I/O devices, the main throttle is the speed of the processor not the access time of devices.

4.1.2 I/O bound

I/O bound threads are threads which perform a lot of I/O actions and therefore have to wait a lot inbetween when it can actually compute.

4.2 Batch / Interactive / Rts scheduling

4.2.1 Batch

"business-world" applications, data analysis. Appropriate for non-preemptive.

The most important metric is throughput.

Comparison of different algorithms

Algorithm	+	-
First-Come/First-Serve (non-preemptive)	Easy, fair	Possibly inefficient (esp. for I/O bound pr
Shortest Job First (non-preemptive)	Optimal for turnaround	Starvation + need to know runtime
Shortest Remaining Time Next (preemptive)	New short jobs get good service	Starvation + need to know runtime

4.2.2 Interactive

It is not known in advance what the system is going to do. Many users that need responsiveness, requiring preemptive scheduling.

Most important metric is response time.

Comparison of different algorithms

Algorithm	+	-
Round-robin	Easy to implement	Easy to get too long or short times per process.
Priority scheduling	Can maximize the throughput easily	Harder to implement
Lottery scheduling	High control over resource use	

4.2.3 Real-time

We will not discuss this during the course, but if these do not run instantly, there is no point in running it. Think car control systems.

for short-lived, short-cycle processes with hard/soft deadlines.

Most important metric is meeting deadlines.

4.3 Processes vs. Threads scheduling

4.4 Multiprocessor hardware

4.5 Multiprocessor scheduling

Lecture 5

Synchronization

5.1 Motivation

If we run different threads in a program and they execute in parallel, this means that they might try to access data at the same time. What happens if Thread 1 tries to read variable A at the same time as Thread 2 tries to write to it? we can't know which one will access the data first.

5.2 The Critical Section problem

When writing syncronized code we create a "critical section" in which only one thread are allowed to execute at a time, even if the critical sections are different only one thread can be in one of them at a time.

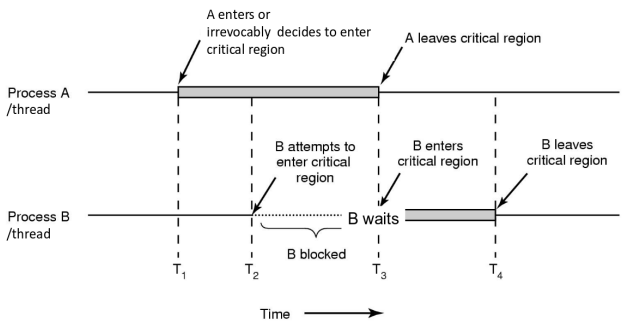


Figure 5.1: The desired outcome of the critical region problem.

We need to solve the problem of critical section in such a way that: only one thread at a time is allowed to execute a critical section; other threads can enter their critical sections in bounded time when no process is currently in its critical section; no thread will be starved, endlessly waiting for its turn of executing a critical section.

5.2.1 Check-competition approach

```
1.  Shared var flagA, flagB: boolean; // indicate A's,
    B's intention to enter; initially both false
2.  Thread A // B is symmetric
3.    repeat
4.      while flagB == true do [nothing]  //busywait
5.      flagA ← true
6.      [critical section]
7.      flagA ← false
8.      [remainder section]
9.    forever
```

Figure 5.2: The check-competition approach to critical section problem.

One approach to the critical section problem is the Check-competition approach (see 5.2), this however fails at the mutual exclusion property, since several threads can manage to execute its line 4 before any thread manages to execute its line 5, blocking threads from executing line 4.

5.2.2 The check-order / after-you approach

```
1.  Shared var turn: (idA..idB)   // initially e.g. idA
    // indicates whose turn it is to enter CS
2.  Thread A  // B is symmetric
3.    repeat
4.      while turn ≠ idA do [nothing]  //busywait
5.      [critical section]
6.      turn ← idB
7.      [remainder section]
8.    forever
```

Figure 5.3: The check-order approach to critical section problem.

Another approach is called check order (see 5.3) and lets the threads take turn executing their critical section, instead waiting for their turn instead of deciding themselves when it's their time to execute. A problem with this however is starvation, since if the thread before never enters its critical section, all following threads will be stuck waiting forever.

5.2.3 Peterson's Algorithm

If we however combine the outcomes of these two approaches, we get a solution that works, Peterson's algorithm is this solution (see 5.4)

```
1.  Shared var turn: (idA..idB) // initially e.g., idA
2.  Shared var flagA, flagB: Boolean // initially false

3.  thread A // B is symmetric
4.   repeat
5.      flagA ← true
6.      turn ← idB
7.      while (flagB and turn == idB) do [nothing]
8.      [critical section]
9.      flagA← false
10.     [remainder section]
11.  forever
```

Figure 5.4: Peterson's algorithm, a working solution to the critical section problem.

5.3 Synchronization and hardware support

One hardware solution to the mutual exclusion problem is to just disable interrupts while in the critical section. There are however some problems with this.

- If we use a multiprocessor system, the threads running on different cores might still cause issues.
- If we run on a single-core system, nothing else can execute, meaning everything else will function.

5.3.1 Read-Modify-Write instructions

We can use an instruction called TestAndSet which atomically tests a value, and sets its value to true. Using this we can create locks which only one process are able to pass through.

```
TestAndSet (aka TAS) Instruction Definition:
boolean TestAndSet (boolean *target) {
  boolean rv = *target;
  *target := true;
  return rv} // Executed atomically by the HW
```

Figure 5.5: Pseudo code for what Test and set does.

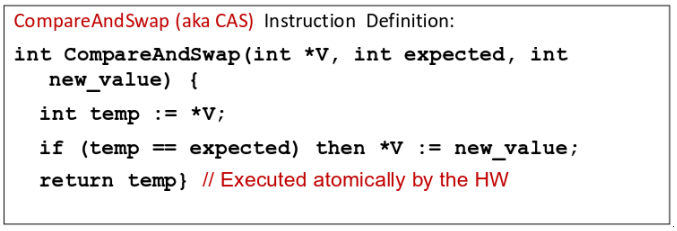


Figure 5.6: Pseudo code for what compare and swap does.

5.3.2 Reflection on busy-waiting

Right now all the solutions are using busy-waiting, meaning they actively check for when they can continue. While this is a simple solution, it consumes computing resources without any reason to, it might starve , and deadlocks might happen if the thread waiting to use its critical section has higher priority than the one in a critical section, meaning the busy-waiting thread gets to execute its wait while the thread it waits for is not allowed to execute until the waiting thread has terminated.

5.4 Semaphores

5.5 Other techniques

5.6 Bounded-buffer producer/consumer

5.7 Resource allocation, deadlocks and necessary conditions for deadlocks

Can we say something about if a certain number of threads could deadlock, without knowing anything about the code?

5.8 dining philosophers

5.9 Lamport's bakery algorithm

5.10 Reader's/Writer's problem

5.11 Os as arbitrator for deadlock avoidance/recovery

Lecture 7

Memory management

7.1 Background and basic hardware

There exist a memory hierarchy inside of our computer; the ones very close to the CPU, i.e. registers, main memory, caches, can be accessed directly by the CPU, the ones further away needs to be accessed through different types of hardware and therefore often take more time.

Registers and caches are fastest to access data from, usually within one cycle, while main memory take many cycles, and all other media take a lot longer.

While there is no actual need to copy things from slow devices to faster ones, like copying files you access from the disk to main memory, or use registers for computations, is not necessary, the amount of time reduction is so impactful that in practice this is seen as necessary.

7.2 Base and limit register

Since all processes needs access to their own memory and will probably use it heavily, it is not feasible to wake up the operating system and chech whether every single process is allowed to write or read from/to memory each time this is done, we need a way to keep track of where each process is allowed to read and write accessible without the OS.

For each process, the CPU keeps track of the first address every process are allowed to access as well as the number of addresses they can access. This data is stored in registers and loaded when context switching.

With these registers, it is easy for the hardware to for every memory access check whether the address requested is within the range reserved for the process, if it is inbetween it gets access, otherwise the OS is called upon.

7.3 Logical vs Physical addresses

Since processes often use hard-coded memory addresses in memory to read and store data, the operating system translates the addresses the process is using into addresses in memory, the address itself might be vastly different.

Using logical instead of physical addresses also help when swapping, since it doesn't matter if the data is swapped into a different memory range.

7.3.1 Memory management Unit - Relocation Register

One version of this is for example if a process uses addresses 0 - 1000, we add a number, maybe $1000 * \text{ps\_id}$ we lets all processes use the same virtual addresses but different logical addresses.

7.4 Swapping

Swapping means moving data back from main memory to store on disk. Usually the main memory is quite small, but the amount of logical addresses in the main memory is vastly larger, and if we keep loading more and more data into main memory we are allowed to do this and the OS is able to handle this. This is because as memory is stopped being accessed, the OS moves that data from main memory into disk until it is later needed.

7.4.1 Which processes can we swap?

It is generally a bad idea to swap the memory of processes in the ready queue, since they could soon be called upon, forcing us to move the data back. Instead swapping the data of processes waiting on I/O, preferrably processes waiting for slower I/O and where the result of the I/O process can be buffered by the OS.

7.5 Memory allocation

Memory contains the OS as well ass the user's processes, we need a way to allocate memory efficiently, preferrably also disallowing processes from writing and reading to the OS memory as well as maybe disallowing reading from certain parts of their own memory.

7.5.1 Contiguous allocation

We allocate each process directly after each other, to translate the physical addresses we just add the offset between the start of logical and physical addresses. Issues with this method is that every time a process terminate or swap, we create a hole, OS need to keep track of holes for placement of processes later.

Contiguous allocation may lead to external fragmentation, where the amount of memory not used is enough for a process, but there not one segment of memory large enough to fit a process. It is possible to defragment the memory but this is not really done for main memory since this takes a lot of time and while doing that it is impossible to use the memory, all processes need to pause execution.

7.5.2 Segmentation

Instead of each process having a block of memory, leading to holes when removed, we can instead allocate different segments in different memory slots. We instead refer to the memory as which segment to access and the position within the segment we want to access.

7.5.3 Paging

We define the logical memory in terms of pages and the physical memory in terms of frames. All pages and frames are the same size, and each time we load a page, we **at least** need to load 4 frames. To keep track of the pages we have a page table in each process.

For the hardware to translate a logical address into a physical, we have a page size which is a power of 2, meaning the lowest n bits define the offset within a frame, and all the other bits are used to define the page number.

When a page is accessed by a process, the hardware looks for it in the page table, if it is not there, an interrupt is created and the operating system loads it.

Lecture 8

Virtual memory

8.1 Virtual memory in a nutshell

8.2 Demand paging

Technically we only need a page in memory during the time we access it. By only loading pages into memory when needed we can save time.

To do this we need new functionality in the mmu.

If we try to access a page and it is in memory we can just access it, if the page is not in memory we call upon the operating system

8.2.1 Valid-Invalid bit

To understand whether the needed page is in memory, we add extra information to the page table, one of them is the valid-invalid bit which represents whether a page is in main memory or in swap.

8.3 Copy-on-Write

When a process spawn another process, both processes copy the page table, and mark all pages as read only. By doing this we only need to allocate new pages as one or both of the processes tries to write to that specific page.

8.4 Page replacement

How do I efficiently load the pages I need? There's different algorithms that help define which pages to load/unload and when.

8.4.1 FIFO

If all frames are used, free the first one loaded into memory. This suffers from an anomaly where sometimes adding more pages leads to more page faults.

8.4.2 Optimal algorithm - Least recently used

Optimally we want to replace the page that will not be used for the longest amount of time, but how? Knowing for certain can be very hard since it is not yet possible to know the future.

Since we cannot know the future, a good prediction can be to look at the past to predict the future. Logically the page that has been unused for the longest amount of time will likely be the one not used for the longest amount of time in the future.

This is the best known algorithm so far.